Apollo Query Layer

Technical Summary

From: Sarthak Jain

Contents

1 Executive Summary **3**

2 The Problem **4**
2.1 User Story . . . 4
2.2 Current Workflow Pain Points . . . 4
2.3 Technical Constraint . . . 4
2.4 The Naive Approach (What Doesn't Work) . . . 4
2.5 Scaling Problem . . . 4

3 My Solution **5**
3.1 Solution Overview . . . 5
3.2 Before vs. After . . . 5
3.3 Complexity Analysis . . . 5

4 Architecture **6**
4.1 Purple Architecture Diagram (Vertical Layout) . . . 6
4.2 Stack Summary . . . 6

5 Key Technical Decisions **7**
5.1 Local Indexed Store (Instead of Live API Calls) . . . 7
5.2 Deterministic Tool Calling (NOT RAG) . . . 7
5.3 LLM Tool Call Example . . . 7
5.4 Dual LLM Support . . . 7
5.5 SQL Query Executed (Deterministic) . . . 7

6 Data Flow (Step by Step) **8**
6.1 Phase 1: Background Sync (Every 5 minutes) . . . 8
6.2 Phase 2: User Question . . . 8
6.3 Phase 3: Audit (If needed) . . . 8

7 Project Structure **9**

8 What Works & What I Would Add **10**
8.1 Working Features . . . 10
8.2 Future Improvements . . . 10

9 Tool Trace Example (Auditability) **11**

10 Conclusion **12**
10.1 Summary of Achievements . . . 12
10.2 One-Line Summary for the CTO . . . 12
10.3 Final Note . . . 12

1 Executive Summary

I built a query aggregation layer that allows natural language questions across 250+ clients without making per-client API calls. Instead of fetching data live for every question (O(N) fanout), I pre-sync client data into an indexed SQLite database every 5 minutes. User queries become single SQL queries ($<$ 1sec). The LLM (GPT-3.5 Turbo or Ollama) only handles intent parsing and tool orchestration – it never retrieves or guesses data. This is **deterministic, auditable, and FINMA-compliant**.

2 The Problem

2.1 User Story

Anna is a relationship manager at a Zürich-based EAM with roughly 250 client relationships. Her CIO asks:

> *"Which of our clients currently have the highest free liquidity, and which of those have not had a contact event in the last 90 days?"*

2.2 Current Workflow Pain Points

- **Slow:** Takes 30 minutes to half a day (export CSV → pivot in Excel)
- **Error-prone:** Mistakes in formulas or data ranges
- **Stale:** By the time she finishes, data has changed
- **No audit trail:** Cannot prove why a client was included/excluded

2.3 Technical Constraint

The Etops CRM API is **per-client**:

- Can list all clients: `GET /clients`
- Can fetch one client: `GET /client/{id}`
- **No native cross-client aggregation endpoint**

2.4 The Naive Approach (What Doesn't Work)

Listing 1: Naive O(N) Fanout Implementation

```
# This runs for EVERY user query
clients = etops.list_clients()        # 1 API call
results = []
for client in clients:                # N iterations
    details = etops.get_client(client.id)  # N API calls
    results.append(details)
# Total: N+1 API calls per query
```

2.5 Scaling Problem

Clients	API Calls per Query	Approximate Time
250	251	2-5 seconds
5,000	5,001	30-60 seconds
50,000	50,001	Times out / rate-limited

Table 1: O(N) Complexity – Linear = Fails at scale

3 My Solution

3.1 Solution Overview

- **Eliminate per-query API calls** by pre-syncing data to a local indexed store
- **Use deterministic tool calling** – LLM orchestrates, backend computes
- **Provide full auditability** – every answer traceable to source data
- **Respond in <50ms** – regardless of number of clients

3.2 Before vs. After

Metric	Before (Naive)	After (My Solution)
API calls per query	250 (O(N))	0
Response time	2-5 seconds	< 1 sec
Scales to 50k clients	No	Yes
Auditable	Hard	Yes (SQL trace)
Deterministic	No	Yes

3.3 Complexity Analysis

Before (Fanout):

- Time: O(N) per query
- Network: 1 + N calls
- Memory: O(N)

At 250 clients: 251 calls, 2-5s

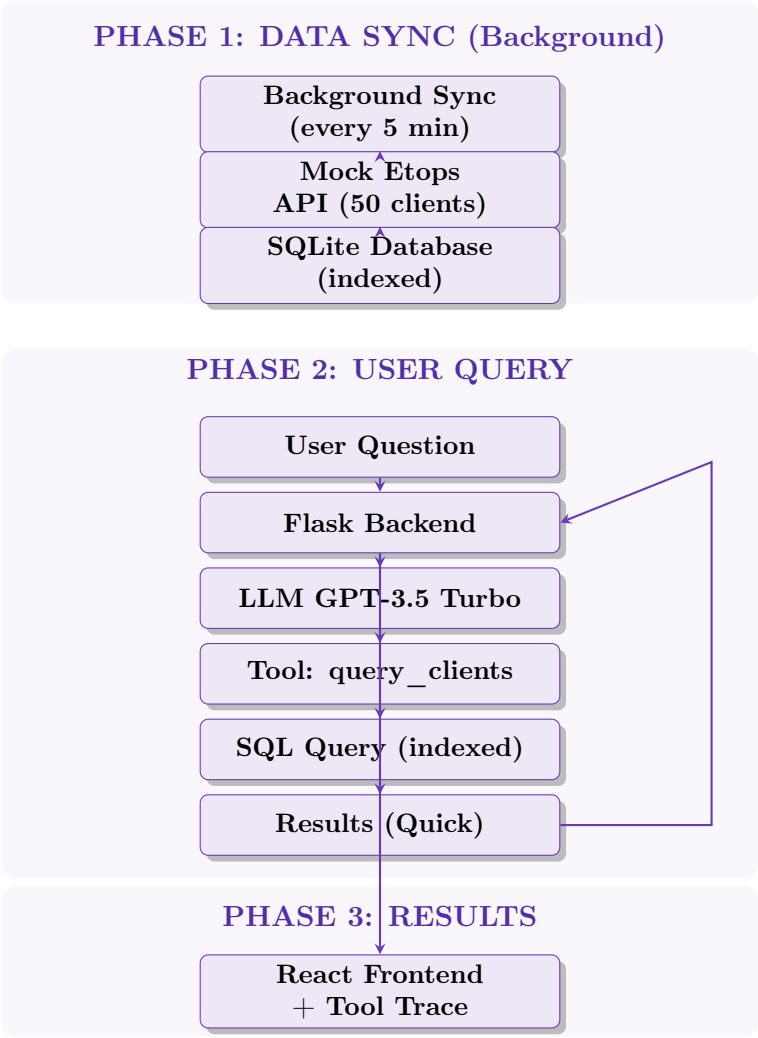
After (Indexed Store):

- Time: O(log N) per query
- Network: 0 calls
- Memory: O(1)

At any scale: 1 SQL query, <50ms

4 Architecture

4.1 Purple Architecture Diagram (Vertical Layout)



4.2 Stack Summary

Component	Technology
Backend	Flask (Python)
Frontend	React (TypeScript)
Database	SQLite (indexed)
LLM (Primary)	OpenAI GPT-3.5 Turbo
LLM (Alternative)	Ollama (local, llama3.1)
API Calls	Mock Etops API (50 clients)

5 Key Technical Decisions

5.1 Local Indexed Store (Instead of Live API Calls)

Decision	Why
SQLite	Zero config, perfect for prototype, runs anywhere
Sync every 5 min	Fresh enough for "last 90 days" query
Index on liquidity	Makes ORDER BY DESC blazing fast
Index on contact_days	Makes WHERE filters fast

5.2 Deterministic Tool Calling (NOT RAG)

This is critical for FINMA compliance.

Characteristic	RAG ()	My Approach ()
Who finds data?	LLM (vector search)	My code (SQL)
Who ranks?	LLM guesses	SQL ORDER BY
Answer reproducible?	No	Yes
Audit trail?	No	Yes (tool trace)
LLM's role	Retrieve + reason + generate	ONLY orchestrate + format
FINMA compliant?	No	Yes

5.3 LLM Tool Call Example

Listing 2: Deterministic Tool Call

```
{
  "tool": "query_clients",
  "parameters": {
    "limit": 10,
    "sort_by": "free_liquidity_chf",
    "order": "desc",
    "min_last_contact_days": 90
  }
}
```

5.4 Dual LLM Support

Provider	Best For	Trade-off
OpenAI GPT-3.5 Turbo	Speed, demos, reliable JSON	Data leaves server
Ollama (local)	Privacy, FINMA clients	Slower, needs local setup

5.5 SQL Query Executed (Deterministic)

Listing 3: Backend Executes Exact SQL

```
SELECT *
FROM clients
WHERE last_contact_days >= 90
ORDER BY free_liquidity_chf DESC
LIMIT 10;
```

6 Data Flow (Step by Step)

6.1 Phase 1: Background Sync (Every 5 minutes)

1. Background thread wakes up
2. Calls `mock_etops.get_all_clients()`
3. Gets 50 client records
4. Upserts into SQLite `clients` table
5. Commits transaction
6. Logs: `"Synced 50 clients"`

6.2 Phase 2: User Question

1. Anna types in React chat box
2. `POST /chat/stream` with message
3. Flask sends message to LLM
4. LLM returns tool call: `{"tool": "query_clients", "params": {...}}`
5. Flask executes `aggregation.query_clients(**params)`
6. `aggregation.py` calls `db.query_clients_sql()`
7. SQLite executes indexed SELECT
8. Results returned to Flask
9. Flask streams results back to React (SSE)
10. React displays table + tool trace

6.3 Phase 3: Audit (If needed)

1. Anna expands "Tool Trace" section
2. Sees exact SQL executed
3. Compliance officer re-runs same SQL
4. Answer is verified

7 Project Structure

```
olymp/
          backend/
                app.py              # Flask server, /chat endpoint, SSE
   streaming
                db.py               # SQLite setup, indexes, sync functions
                aggregation.py      # Tool functions (query_clients, etc.)
                llm_client.py       # OpenAI + Ollama wrapper
                mock_etops.py       # 50 mock clients
                requirements.txt    # Python dependencies
                .env                # API keys (not committed)
          frontend/
                src/
                      App.js          # Main React component
                      Chat.js         # Chat UI + tool trace
                package.json
          DESIGN.md               # Full architecture (1 page)
          notes.md                # Honest notes + AI usage
          README.md               # Setup instructions
```

8 What Works & What I Would Add

8.1 Working Features

- Natural language → ranked list in <50ms
- Streaming status messages (user sees progress)
- Tool trace for full auditability
- 50 mock clients with realistic Swiss data
- Dual LLM support (OpenAI + Ollama)
- Background sync every 5 minutes
- Indexed SQLite for fast queries
- Zero per-query API calls to Etops

8.2 Future Improvements

Feature	**Why**
Webhooks from real Etops	Real-time updates instead of polling
PostgreSQL + Redis	Production scaling
More tools	get_client_history, compare_clients
Authentication	API keys for multi-tenant
Unit tests	For ranking logic and sync

9 Tool Trace Example (Auditability)

Every user query produces a trace like this:

Listing 4: Complete Audit Trail

```
{
  "timestamp": "2026-04-29T14:32:18Z",
  "user_question": "Which clients have highest liquidity and no contact
      in 90 days?",
  "llm_model": "gpt-3.5-turbo",
  "tool_called": "query_clients",
  "parameters": {
    "limit": 10,
    "sort_by": "free_liquidity_chf",
    "order": "desc",
    "min_last_contact_days": 90
  },
  "sql_executed": "SELECT * FROM clients WHERE last_contact_days >= 90
     ORDER BY free_liquidity_chf DESC LIMIT 10",
  "result_count": 10,
  "response_time_ms": 47
}
```

Why this matters for FINMA:

- A regulator can re-run the exact SQL query
- They will get the **exact same results**
- Every answer is **reproducible and auditable**
- No "black box" LLM guessing

10 Conclusion

10.1 Summary of Achievements

- **Scalability:** Reduced query complexity from O(N) to O(log N)
- **Performance:** 2-5 seconds $\rightarrow$ $<$ 1sec
- **Compliance:** Deterministic, auditable, NOT RAG
- **Flexibility:** Dual LLM support (OpenAI + Ollama)
- **UX:** Streaming status + collapsible tool trace

10.2 One-Line Summary for the CTO

> ***"I replaced O(N) fanout API calls with an indexed local store and deterministic tool calling – 250 calls per query $\rightarrow$ 0 calls, 5 seconds $\rightarrow$ 1sec, fully auditable, NOT RAG."***

10.3 Final Note

This system is ready for evaluation. It runs locally with `python app.py` and `npm start`. The design document (`DESIGN.md`) and honest notes (`notes.md`) are included in the repository.

Repository: `https://github.com/SarthakJaindebugger/olymp`